

Rapport d'audit technique - Wildfire

Mesures au 28 septembre 2026 • Révision du 29 septembre 2026

Dépôt GitHub du projet : <https://github.com/Antoine-Ferron/jdv-opti>

Description fonctionnelle. Wildfire simule un incendie sur une grille torique : sur un décor de terrain et de vent fixes, combustion et repos évoluent selon des règles synchrones de voisinage. L'audit compare six moteurs Go et un volet indépendant de sérialisation et de persistance. Règles du système ([../internal/fire/REGLES.md](#)).

Partie I - Reporting, métrologie & résultats

1.1 Bancs d'essai et protocole

Composant	Banc A - référence	Banc B - témoin
CPU / RAM exposée	Apple M1, 8 cœurs / 8 Gio	Intel Core Ultra 9 275HX, 24 cœurs / 31 Gio
Caches exposés	L1d 64 Kio, L1i 128 Kio, L2 4 Mio ; ligne 128 o	L1d 48 Kio, L1i 64 Kio, L2 3 Mio, L3 36 Mio ; ligne 64 o
Système	macOS 27.0, arm64	Ubuntu 26.04 sous WSL2, amd64
Runtime	Go 1.27.1	Go 1.27.1, GOAMD64 v1

Protocole CPU : dix échantillons Go, résumés par benchstat ; quinze exécutions Hyperfine, après trois chauffes, sans shell intermédiaire. Les temps Go sont des **médianes**, ceux d'Hyperfine des **moyennes**. Les « ± » représentent respectivement un IC à 95 % et un écart-type. Le CV (écart-type / moyenne) signale la dispersion, pas la significativité.

Mesure	Travail chronométré
Hyperfine, binaire <code>-quiet</code>	Démarrage + génération de carte + construction + simulation ; ni rendu, ni I/O
BenchmarkRun	Construction + au plus 50 tours, hors génération ; état initial recréé à chaque opération
BenchmarkStep	Tours successifs après 50 tours de chauffe ; ni construction, ni <code>Burning</code>

L'état continue d'évoluer pendant `Step` : ses gains ne décrivent pas exactement les 500 premiers tours du binaire. Les six moteurs sont comparés dans une même campagne par banc. La baseline A atteint **5,4 % de CV**, au-delà du seuil d'alerte de 2 % : prudence sur les petits gains. Les différences entre bancs ne sont pas attribuables au seul processeur.

Statistiques baseline / moteur retenu, binaire 1024², 64 foyers, 500 tours, campagne 981c735, 15 répétitions par ligne. Écart-type et variance calculés sur les échantillons avec le diviseur $n - 1$, avant arrondi.

Banc / moteur	Moyenne (s)	Médiane (s)	Écart-type (s)	Variance (s ²)	CV
A / naive	6,6328	6,5498	0,3588	1,288e-1	5,4 %
A / bitpack	0,5916	0,5897	0,0077	5,956e-5	1,3 %
B / naive	8,2452	8,2317	0,0564	3,186e-3	0,7 %
B / bitpack	0,7661	0,7675	0,0080	6,329e-5	1,0 %

Sources : [environnement A \(../results/981c735/m1-air/env.md\)](#) / [B \(../results/981c735/x86-controle/env.md\)](#) ; [benchmarks \(../internal/bench/bench_test.go\)](#) ; statistiques des six moteurs [A \(../results/981c735/m1-air/hyperfine-stats.md\)](#) / [B \(../results/981c735/x86-controle/hyperfine-stats.md\)](#).

1.2 Charge et dimensionnement

Grille	Cases	État mutable de la baseline, à 2 o/case
256 ²	65 536	128 Kio
1024² - référence	1 048 576	2 Mio
2048 ²	4 194 304	8 Mio

À 1024², la grille seule tient sous les capacités L2 exposées ; avec terrain, vent et tampon d'ignition, la baseline manipule environ **5 Mio de tableaux**. Aucun compteur de défauts de cache n'a été mesuré.

Deux charges : **embrasement**, graine 42 et 64 foyers initiaux ; **front**, graine 42, un foyer et 50 premiers tours. Ces noms ne garantissent pas une densité constante.

1.3 Comparaison des moteurs

Binaire complet : 1024², 64 foyers, 500 tours demandés, campagne 981c735. Temps moyens ; CV entre parenthèses ; accélérations par rapport à naive.

Moteur	Stratégie	Banc A	Banc B	Accélération A / B
naive	[[[]Ce11, tampon alloué par tour	6,6328 s (5,4 %)	8,2452 s (0,7 %)	×1 / ×1
flat	Grilles contiguës, tampons réutilisés	6,3781 s (0,8 %)	8,2330 s (0,6 %)	×1,040 / ×1,001
counters	Compteur de cases en feu	6,0149 s (1,7 %)	7,8536 s (0,8 %)	×1,103 / ×1,050

Moteur	Stratégie	Banc A	Banc B	Accélération A / B
ghost	Halo, propagation sans modulo	5,0866 s (0,4 %)	4,5822 s (1,2 %)	×1,304 / ×1,799
bitpack	Plans de bits, calcul par mots	0,5916 s (1,3 %)	0,7661 s (1,0 %)	×11,21 / ×10,76
front	Listes de cases actives	9,0816 s (1,8 %)	8,2312 s (0,4 %)	×0,730 / ×1,002

À 1024², le microbenchmark mesure **1 Mio et une allocation par Step pour naive**, contre **zéro pour tous les moteurs optimisés**. flat n'est pas une régression établie ici.

Filiation : flat reprend naive, counters ajoute le compteur, ghost remplace l'adressage de propagation, puis bitpack remplace représentation et halo. **front est une variante de counters, pas un successeur de bitpack.**

Step/embracement/1024	naive	bitpack	Accélération
Banc A	10 279,7 µs	196,9 µs	×52,2
Banc B	12 531,2 µs	173,3 µs	×72,3

Charge creuse : Run/front/1024, construction incluse, génération exclue.

Banc	counters	bitpack	front	Gain de front vs counters / bitpack
A	122,866 ms	12,275 ms	2,161 ms	×56,9 / ×5,68
B	84,770 ms	11,869 ms	3,737 ms	×22,7 / ×3,18

L'objectif de front (au moins 20 % de temps en moins que counters, zéro allocation/Step) est atteint. En contrepartie, Run alloue **43 Mio/op contre 5 Mio/op**, soit **×8,6** : trois listes préallouées et l'état réservent environ 43N octets sur ces machines 64 bits, hors décor. **Ce n'est pas une mesure du pic de RAM du processus.**

Sources : Hyperfine [A \(../results/981c735/m1-air/hyperfine.json\)](#) / [B \(../results/981c735/x86-controle/hyperfine.json\)](#) ; Go brut [A \(../results/981c735/m1-air/bench.txt\)](#) / [B \(../results/981c735/x86-controle/bench.txt\)](#) ; [contrat de front \(../internal/front/README.md\)](#).

1.4 Sérialisation et persistance

Formats, banc B, 1024², campagne ae8b974, dix répétitions, avant le pool. État préconstruit, écriture vers io.Discard : ni capture, ni disque, ni réseau.

Format	Médiane d'écriture (IC 95 %)	Taille	Réduction de taille vs JSON
JSON verbeux	534,7 ms (±28 %)	120,4 Mo	×1

Format	Médiane d'écriture (IC 95 %)	Taille	Réduction de taille vs JSON
Protobuf	13,84 ms (± 3 %)	4,19 Mo	$\times 28,7$
Binaire packed	1,338 ms (± 1 %)	1,573 Mo	$\times 76,6$
packed, décor omis	0,849 ms (± 1 %)	524,3 ko	$\times 229,7$

Le résultat vaut pour les schémas implémentés : JSON contient un objet par case avec coordonnées et indentation ; packed range deux états par octet. Omettre le décor exige de le conserver et de le rattacher ailleurs, ce que le codec n'automatise pas. Protobuf pourrait également omettre le décor ou transporter des données bit-packées.

PostgreSQL 17.11 local, banc B, campagne 8a71072, six répétitions. Durabilité désactivée : `fsync=off`, `synchronous_commit=off`, `full_page_writes=off`.

Essai	Référence	Variante	Accélération
500 INSERT unitaires → COPY	83,35 ms	6,565 ms	$\times 12,7$
5 000 INSERT unitaires → COPY	1,033 s	54,38 ms	$\times 19,0$
Index, 20 000 lignes, empreintes uniques	601,8 μ s	153,9 μ s	$\times 3,9$
Index, 500 000 lignes, empreintes uniques	8 257,2 μ s	176,5 μ s	$\times 46,8$
Index, 500 000 lignes, empreinte présente dans 10 % des lignes	8 776 μ s	4 567 μ s	$\times 1,9$

Les insertions concernent des métadonnées, pas les snapshots. COPY change à la fois le protocole, le traitement des commandes et les transactions (INSERT en autocommit) : le gain ne mesure pas la seule latence réseau. Les IC des insertions atteignent ± 16 %. Les requêtes répétées après ANALYZE ne sont pas des tests à cache froid ; ces temps ne décrivent pas une production durable.

Sources : [formats \(../results/ae8b974/x86-controle/benchstat.txt\)](#) ; [SQL \(../results/8a71072/x86-controle/benchstat-store.txt\)](#) ; [configuration SQL \(../results/8a71072/x86-controle/env.md\)](#).

1.5 Reproductibilité

```
make tools          # benchstat ; Go et Hyperfine requis
make test
make bench-cpu BANC=m1-air    # adapter le nom au banc
make profile IMPL=bitpack BANC=m1-air
make db              # base expérimentale
make bench-io BANC=m1-air    # résultats bruts Snapshot/Store
make db-stop        # arrête la base et supprime ses données
# Pool : package distinct, non parcouru par bench-io
go test ./internal/snapshot -run '^$' -bench '^BenchmarkPool$' -benchmem -count 6
```

Le [script CPU \(../scripts/run_benchmarks.sh\)](#) archive sous `results/<commit-de-code>/<banc>/` et contrôle les modifications suivies, sauf forçage ; `bench-io` n'a pas ces mêmes garanties. Les essais SQL sont sautés si la base est inaccessible. Certaines fiches signalent un arbre modifié, sans diff archivé : l'identité exacte du code mesuré ne peut alors être certifiée (le signal peut aussi venir de la réécriture

des résultats). Une reproduction stricte demande des commandes complètes, un arbre identifié et des versions figées de benchstat et PostgreSQL.

Partie II - Diagnostic & choix d'ingénierie

2.1 Diagnostic du chemin critique

Trois coûts sont identifiés dans la baseline :

Observation	Mesure	Levier
Tampon d'ignition recréé	1 Mio alloué par tour à 1024 ²	Réutiliser les tampons
Adressage torique	Mod : 5,24 % du CPU propre sur A, 41,06 % sur B (dcc3622)	Halo puis opérations sur mots
Recompte de toutes les cases	Burning : 20,68 % sur A, 23,39 % sur B en charge creuse (1d3a093)	Compteur incrémental

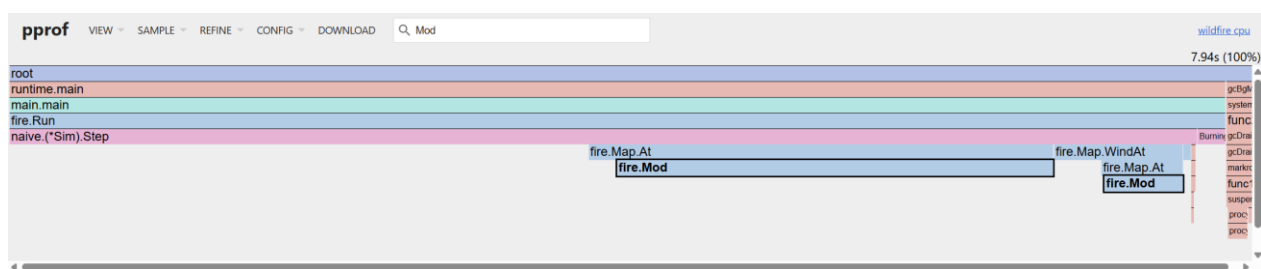


Figure 2.1 - Baseline B, campagne dcc3622 : `Mod` représente 41,06 % du temps propre, 41,18 % du cumulé. Le profil CPU démarre après la génération de carte.

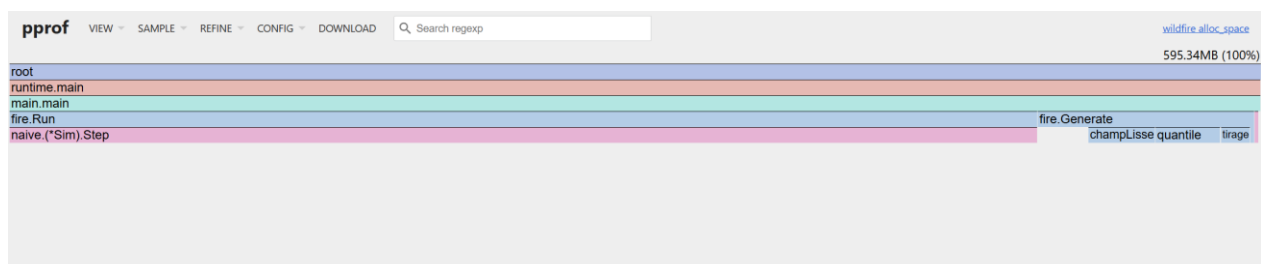


Figure 2.2 - Baseline A, campagne dcc3622, profil `alloc_space` : environ **489 Mio attribués à Step**, soit **82,14 %** des allocations cumulées échantillonnées. Le « MB » affiché par pprof correspond ici à 2²⁰ octets ; il ne mesure ni le pic de RAM ni le temps du GC.

Ce profil mémoire couvre le démarrage, contrairement au profil CPU ; son échantillonnage est affiné après `Generate`. Le microbenchmark confirme le levier : à 1024², `naive` alloue 1 Mio par `Step`, contre zéro pour `flat` et les moteurs suivants.

Ces parts localisent les coûts, sans établir un nombre de cycles par modulo, un défaut de cache ou un GC dominant. Les temps cumulés incluent les appels : ils ne s'additionnent pas aux temps propres. `Map.At` calcule un indice torique ; l'indirection `[[[]]Cell` est dans la grille.

Sources : profils CPU [A \(../results/dcc3622/m1-air/profiles/naive-cpu-top.txt\)](#) / [B \(../results/dcc3622/x86-controle/profiles/naive-cpu-top.txt\)](#) ; profils creux [A \(../results/1d3a093/m1-air/profiles/naive-front50-cpu-top.txt\)](#) / [B \(../results/1d3a093/x86-controle/profiles/naive-front50-cpu-top.txt\)](#) ; profil d'allocations [A \(../results/dcc3622/m1-air/profiles/naive-mem-top.txt\)](#).

2.2 Choix d'ingénierie en trois axes

1. Mémoire et organisation du calcul. `flat` réutilise les tampons ; `counters` évite le balayage de `Burning` ; `ghost` supprime les modulus de propagation. `bitpack` représente l'état sur **4 bits au lieu de 16 par case**, hors arrondi des lignes, et traite **64 cases par mot**, avec un traitement séparé du vent. Cela ne divise pas toute la mémoire par quatre : `New/bitpack` alloue 5,437 Mio, contre 5,000 pour `counters`, hors carte. `front` réduit plutôt le nombre de cases visitées ; son pire cas et sa mémoire restent $O(N)$.

2. Concurrence et scalabilité CPU. Axe non implémenté : aucune courbe de montée en charge par nombre de workers. L'arrêt à l'extinction existe déjà dans `fire.Run`. Le poids des coûts fixes oriente la prochaine expérience vers la génération, sans rendre la parallélisation de `Step` inutile pour des simulations plus longues.

3. I/O et persistance. Compacter les états, grouper les insertions et indexer les recherches sélectives répondent à des coûts distincts. Ces gains ne s'ajoutent pas au $\times 11$ du moteur : le binaire mesuré n'utilise pas cette chaîne I/O.

2.3 Journal expérimental et confrontation critique

Trois exemples relient les hypothèses documentées aux expériences et aux décisions retenues dans cet audit. Les objectifs chiffrés des moteurs viennent de leurs README ; l'hypothèse de déduplication est décrite dans le prototype.

Hypothèse initiale	Expérience choisie	Résultat	Décision
bitpack : calculer par mots pour réduire d'au moins 20 % le temps de <code>Step</code> face à <code>ghost</code> , sans allocation	<code>Step/embracement/1024</code> , dix répétitions sur A et B ; contrôle Hyperfine à 500 tours	Temps de <code>Step</code> divisé par 45,5 sur A et 44,4 sur B ; zéro allocation ; gain global confirmé au §1.3	Objectif atteint ; retenir <code>bitpack</code> pour la charge 500 tours / 64 foyers
front : ne visiter que les cases actives pour réduire d'au moins 20 % <code>Run/front</code> face à <code>counters</code> , sans allocation par <code>Step</code>	<code>Run/front/1024</code> , 50 tours, dix répétitions sur A et B ; allocations et contre-essai à 500 tours / 64 foyers	Temps divisé par 56,9 sur A et 22,7 sur B , mais $\times 8,6$ d'octets alloués par Run et régression au contre-essai	Conserver comme variante creuse ; ne pas remplacer <code>bitpack</code> sur la charge dense longue

Hypothèse initiale	Expérience choisie	Résultat	Décision
Déduplication : éviter les sérialisations répétées pour gagner du temps malgré le hachage	Sur B, 201 états préconstruits en 128 ² ; cache de 0 à 32 entrées, six répétitions ; hachage inclus, sortie vers <code>io.Discard</code>	Avec 32 entrées, 72,64 % d'écritures évitées ; -55,9 % de temps sur <code>proto</code> , +73,6 % sur <code>packed</code>	Écarter la déduplication pour <code>packed</code> dans cette charge ; bénéfice local pour <code>proto</code> , sans généralisation à un archivage réel

Pourquoi les verdicts diffèrent. Le contre-essai `front` coûte **51,0 % de temps en plus sur A** et **4,8 % sur B** face à `counters` : le succès dépend du régime. Pour la déduplication, la série couvre 2 000 tours, un foyer, avec capture tous les dix tours. Le hachage coûte 27,50 μ s, contre environ 19 μ s d'écriture moyenne `packed` sans cache (3,824 ms / 201). La rentabilité exige : **coût du hachage et de la recherche < taux de doublons $\times$ coût d'écriture évité**. Le prototype ne vérifie pas les collisions et ne conserve pas les références chronologiques : ce n'est pas encore un archivage complet sans perte.

Autres enseignements.

- `flat` supprime les allocations par tour, sans gain temporel important sur le binaire : moins d'allocations ne garantit ni moins de CPU ni moins de mémoire vivante.
- L'index gagne $\times 3,9$ à 20 000 lignes uniques et $\times 46,8$ à 500 000 ; le seuil exact de $\times 10$ reste inconnu. L'`EXPLAIN` à **20 000 lignes** passe de 192 blocs en `Seq Scan` à trois en `Index Only Scan`, avec encore un accès au tas.
- Le pool préchauffé réduit les octets alloués d'environ 99,9 %. Les gains de temps significatifs sont de $-1,87$ % sur A pour `packed` 1024² et de $-6,27$ à $-21,60$ % dans trois cas sur B ($p=0,002$) ; les autres cas sont non significatifs. Les parts du GC et des caches ne sont pas isolées.

Sources : hypothèses [bitpack \(../internal/bitpack/README.md\)](#), `front` ([../internal/front/README.md](#)), `déduplication` ([../internal/snapshot/dedup.go](#)) ; `protocole de série` ([../internal/bench/serie_test.go](#)) ; comparaisons Go A ([../results/981c735/m1-air/benchstat.txt](#)) / B ([../results/981c735/x86-controle/benchstat.txt](#)) ; plans SQL ([../results/8a71072/x86-controle/explain-store.txt](#)) ; pool A ([../results/d533e30/m1-air/benchstat-pool.txt](#)) / B ([../results/935ffbc/x86-controle/benchstat-pool.txt](#)) ; `série` ([../results/935ffbc/x86-controle/benchstat-serie.txt](#)) ; `hachage corrigé` ([../results/935ffbc/x86-controle/empreinte.txt](#)).

2.4 Coûts fixes et loi d'Amdahl

Les gains de `Step` dépassent ceux du binaire, qui inclut notamment la génération. Les compléments Hyperfine donnent la décomposition **approximative** suivante :

bitpack, 1024 ²	Banc A	Banc B
Binaire, 500 tours / 64 foyers	591,6 ms	766,1 ms
Référence hors tours	508,2 $\pm$ 14,5 ms	682,3 $\pm$ 13,1 ms
Différence des moyennes	83,4 ms (14,1 %)	83,8 ms (10,9 %)

La référence A est « bitpack 0 tour » ; celle de B est étiquetée « generation seule », sans commande détaillée archivée. **Zéro tour inclut encore le démarrage et le constructeur dans la CLI** : ce n'est pas une mesure isolée de `Generate`, et les soustractions restent sensibles au bruit.

Amdahl donne $\text{accélération globale} = 1 / ((1 - f) + f / s)$. Si la fraction accélérable vaut 11 à 14 %, la supprimer entièrement réduit le temps total de 11 à 14 % ($\times 1,12$ à $\times 1,16$) ; la diviser par deux ne réduit le total que de 5,5 à 7 %.

L'essai creux B illustre cette limite : `bitpack` → `front` accélère le microbenchmark, mais le binaire passe de **686,8 ± 11,2 à 686,5 ± 11,2 ms**, sans gain discernable. **Ce test ne comporte pas `counters`** : il ne démontre pas l'absence de gain global face à cette base. Les quelques millisecondes résiduelles ne permettent pas non plus de fixer un plafond précis à 0,6 %.

Suite proposée : chronométrer séparément génération, construction et tours, puis compléter `front/counters` sur le binaire creux. Le profil CPU actuel exclut `Generate` : il faut d'abord le mesurer avant de tester la réutilisation des tris de quantiles, la réduction des modulus du lissage ou sa parallélisation bornée. Aucun gain futur n'est présenté comme acquis.

Sources : [zéro tour A \(../results/981c735/m1-air/zero-turns-hyperfine.json\)](https://github.com/0x00000000/981c735/m1-air/zero-turns-hyperfine.json) ; [essai creux B \(../results/981c735/x86-controle/front50-hyperfine.md\)](https://github.com/0x00000000/981c735/x86-controle/front50-hyperfine.md) ; [périmètre de la CLI \(../cmd/wildfire/main.go\)](https://github.com/0x00000000/cmd/wildfire/main.go) ; [générateur \(../internal/fire/carte.go\)](https://github.com/0x00000000/internal/fire/carte.go).

Conclusion

`bitpack` est le meilleur choix mesuré pour 500 tours / 64 foyers : environ $\times 11$ sur les deux bancs. `front` réussit l'optimisation creuse, au prix d'une forte capacité mémoire réservée. Le choix dépend donc de la charge, pas d'un classement universel des moteurs. La CLI conserve `naïve` par défaut.

Le volet I/O confirme l'intérêt du format compact, du groupage et de la sélectivité des index ; la déduplication rappelle que le coût d'une optimisation peut dépasser le travail évité. La priorité suivante est la mesure isolée de la génération ; la scalabilité multicœur reste à démontrer.

Annexe - Gouvernance technique IA

La [constitution du dépôt \(../constitution.md\)](https://github.com/0x00000000/constitution.md) fixe quatre directives :

Directive	Application attendue
Rôle	Ingénieur performance ; correction prioritaire ; baseline et tests de référence figés
Contraintes	Éviter chaînes, allocations injustifiées, goroutines non bornées et verrous par case sur le chemin chaud
Preuve	Hypothèse chiffrée + commande de mesure ; documenter gains, régressions et résultats non significatifs
Format	Réponses compactes, commandes reproductibles, chiffres avec unités

Ce cadre n'est pas un contrôle automatique : la référence de 64 o par ligne de cache ne vaut pas pour A (128 o), et `Fingerprint` conserve du formatage hérité, hors de `fire.Run`.